S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 09

Aim: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Optimal page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

Name: Kashish Vijay Thakre

USN: CM24015

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

- **Aim:** A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Least recently used page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

- **Objectives:**
  1. Implement the Least Recently Used (LRU) page replacement algorithm for memory management.
  2. Calculate the number of page hits and page faults for a given page reference sequence.
  3. Determine the percentage of page hits and page faults based on the system's frame capacity.
- **Requirements:**
  - ✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

  - ✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

- **Theory:**

In an operating system, page replacement refers to a scenario in which a page from the main memory should be replaced by a page from the secondary memory. Page replacement occurs due to page faults. The various page replacement algorithms like FIFO, Optimal page replacement, LRU, LIFO, and Random page replacement help the operating system decide which page to replace.

What is Page Replacement in Operating Systems?

Page replacement is needed in the operating systems that use virtual memory using Demand Paging. As we know in Demand paging, only a set of pages of a process is loaded into the memory. This is done so that we can have more processes in the memory at the same time.

When a page that is residing in virtual memory is requested by a process for its execution, the Operating System needs to decide which page will be replaced by this requested page. This process is known as page replacement and is a vital component in virtual memory management.

Why Need Page Replacement Algorithms?

To understand why we need page replacement algorithms, we first need to know about page faults. Let's see what is a page fault.

Page Fault: A Page Fault occurs when a program running in CPU tries to access a page that is in the address space of that program, but the requested page is currently not loaded into the main physical memory, the RAM of the system. Since the actual RAM is much less than the virtual memory the page faults occur. So whenever a page fault occurs, the Operating system has to replace an existing page in RAM with the newly requested page. In this scenario, page replacement algorithms help the Operating System in deciding which page to replace. The primary objective of all the page replacement algorithms is to minimize the number of page faults.

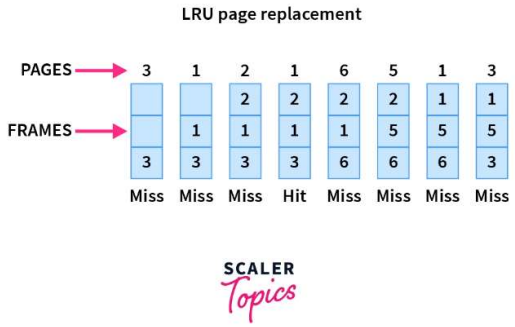
Page Replacement Algorithms in Operating Systems

First In First Out (FIFO)

The FIFO algorithm is the simplest of all the page replacement algorithms. In this, we maintain a queue of all the pages that are in the memory currently. The oldest page in the memory is at the front end of the queue and the most recent page is at the back or rear end of the queue.

Whenever a page fault occurs, the operating system looks at the front end of the queue to know the page to be replaced by the newly requested page. It also adds this newly requested page at the rear end and removes the oldest page from the front end of the queue.

Example: Consider the page reference string as 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames. Let's try to find the number of page faults:



Initially, all of the slots are empty so page faults occur at 3,1,2.
Page faults = 3
When page 1 comes, it is in the memory so no page fault occurs.
Page faults = 3
When page 6 comes, it is not present and a page fault occurs. Since there are no empty slots, we remove the front of the queue, i.e 3.
Page faults = 4
When page 5 comes, it is also not present, and hence a page fault occurs. The front of the queue i.e. 1 is removed.
Page faults = 5
When page 1 comes, it is not found in memory and again a page fault occurs. The front of the queue i.e. 2 is removed.
Page faults = 6
When page 3 comes, it is again not found in memory, a page fault occurs, and page 6 is removed being on top of the queue
Total page faults = 7

Belady's anomaly: Generally if we increase the number of frames in the memory, the number of page faults should decrease due to obvious reasons. Belady's anomaly refers to the phenomena where increasing the number of frames in memory, increases the page faults as well.

Advantages

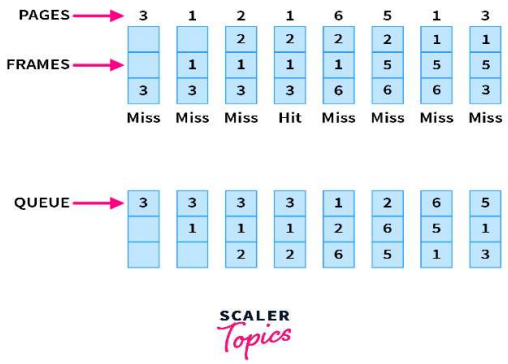
- Simple to understand and implement.
- Does not cause more overhead.

Disadvantages

- Poor performance
- Doesn't use the frequency of the last used time and just simply replaces the oldest page.
- Suffers from Belady's anomaly.
- Optimal Page Replacement in OS

Optimal page replacement is the best page replacement algorithm as this algorithm results in the least number of page faults. In this algorithm, the pages are replaced with the ones that will not be used for the longest duration of time in the future. In simple terms, the pages that will be referred to farthest in the future are replaced in this algorithm.

Example: Let's take the same page reference string 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames as we saw in FIFO. This also helps you understand how Optimal Page replacement works the best.



Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.
Page faults = 3
When page 1 comes, it is in the memory and no page fault occurs.
Page faults = 3
When page 6 comes, it is not in the memory, so a page fault occurs and 2 is removed as it is not going to be used again.
Page faults = 4
When page 5 comes, it is also not in the memory and causes a page fault. Similar to above 6 is removed as it is not going to be used again.
page faults = 5
When page 1 and page 3 come, they are in the memory so no page fault occurs.
Total page faults = 5

Advantages

- Excellent efficiency.
- Less complexity.
- Easy to use and understand.
- Simple data structures can be used to implement.
- Used as the benchmark for other algorithms.

Disadvantages

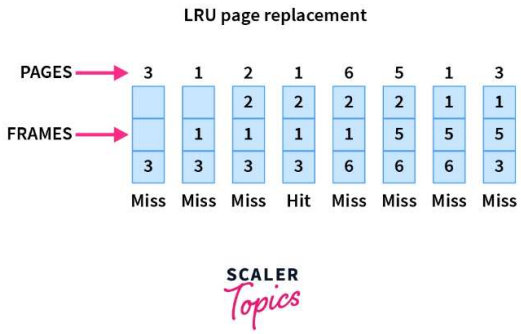
- More time consuming
- Difficult for error handling
- Need future awareness of the programs, which is not possible every time

Least Recently Used (LRU) Page Replacement Algorithm

The least recently used page replacement algorithm keeps the track of usage of pages over a period of time. This algorithm works on the basis of the principle of locality of a reference which states that a program has a tendency to access the same set of memory locations repetitively over a short period of time. So pages that have been used heavily in the past are most likely to be used heavily in the future also.

In this algorithm, when a page fault occurs, then the page that has not been used for the longest duration of time is replaced by the newly requested page.

Example: Let's see the performance of the LRU on the same reference string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:



Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.
Page faults = 3
When page 1 comes, it is in the memory and no page fault occurs.
Page faults = 3
When page 6 comes, it is not in the memory, so a page fault occurs and the least recently used page 3 is removed.
Page faults = 4
When page 5 comes, it again causes a page fault, and page 1 is removed as it is now the least recently used page.
Page faults = 5
When page 1 comes again, it is not in the memory, and hence page 2 is removed according to the LRU.
Page faults = 6

When page 3 comes, the page fault occurs again and this time page 6 is removed as the least recently used one.
Total page faults = 7

Now in the above example, the LRU causes the same page faults as the FIFO, but this may not always be the case as it will depend upon the series, the number of frames available in memory, etc. In fact, on most occasions, LRU is better than FIFO.

Advantages

- It is open for full analysis
- Doesn't suffer from Belady's anomaly
- Often more efficient than other algorithms

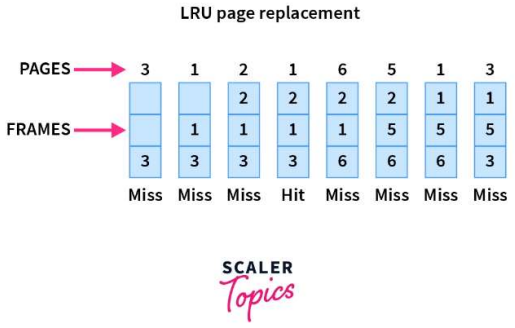
Disadvantages

- It requires additional data structures to be implemented
- More complex
- High hardware assistance is required.

Last In First Out (LIFO) Page Replacement Algorithm

This is the Last in First Out algorithm and works on LIFO principles. In this algorithm, the newest page is replaced by the requested page. Usually, this is done through a stack, where we maintain a stack of pages currently in the memory with the newest page being at the top. Whenever a page fault occurs, the page at the top of the stack is replaced.

Example: Let's see how the LIFO performs for our example string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:



Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs.

Page faults = 3

When page 6 comes, the page fault occurs and page 2 is removed as it is on the top of the stack and is the newest page.

Page faults = 4

When page 5 comes, it is not in the memory, which causes a page fault, and hence page 6 is removed being on top of the stack.

Page faults = 5

When page 1 and page 3 come, they are in memory already, hence no page fault occurs.

Total page faults = 5

As you may notice, this is the same number of page faults as the Optimal page replacement algorithm. So we can say that for this series of pages, this is the best algorithm that can be implemented without the prior knowledge of future references.

Advantages

- Simple to understand
- Easy to implement
- No overhead

Disadvantages

Does not consider Locality principle, hence may produce worst performance.

The old pages may reside in memory forever even if they are not used.

Random Page Replacement in OS

This algorithm, as the name suggests, chooses any random page in the memory to be replaced by the requested page. This algorithm can behave like any of the algorithms based on the random page chosen to be replaced.

Example: Suppose we choose to replace the middle frame every time a page fault occurs. Let's see how our series of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames perform with this algorithm:

Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs.

Page faults = 3

When page 6 comes, the page fault occurs, we replace the middle element i.e 1 is removed.

Page faults = 4

When page 5 comes, the page fault occurs again and middle element 6 is removed

Page faults = 5

When page 1 comes, there is again a page fault, and again the middle element 5 is removed

Page faults = 6

When page 3 comes, it is in memory, hence no page fault occurs.

Total page faults = 6

As we can see, the performance is not the best, but it's also not the worst. The performance in the random replacement algorithm depends on the choice of the page chosen at random.

Advantages

- Easy to understand and implement
- No extra data structure needed to implement

- No overhead

Disadvantages

- Can not be analyzed, may produce different performances for the same series.
- Can suffer from Belady's anomaly.

Summary:

i. The objective of page replacement algorithms is to minimize the page faults
ii. FIFO page replacement algorithm replaces the oldest page in the memory
iii. Optimal page replacement algorithm replaces the page which will be referred farthest in the future
iv. LRU page replacement algorithm replaces the page that has not been used for the longest duration of time
v. LIFO page replacement algorithm replaces the newest page in memory
vi. Random page replacement algorithm replaces any page at random
vii. Optimal page replacement algorithm is considered to be the most effective algorithm but cannot be implemented in practical scenarios due to various limitations

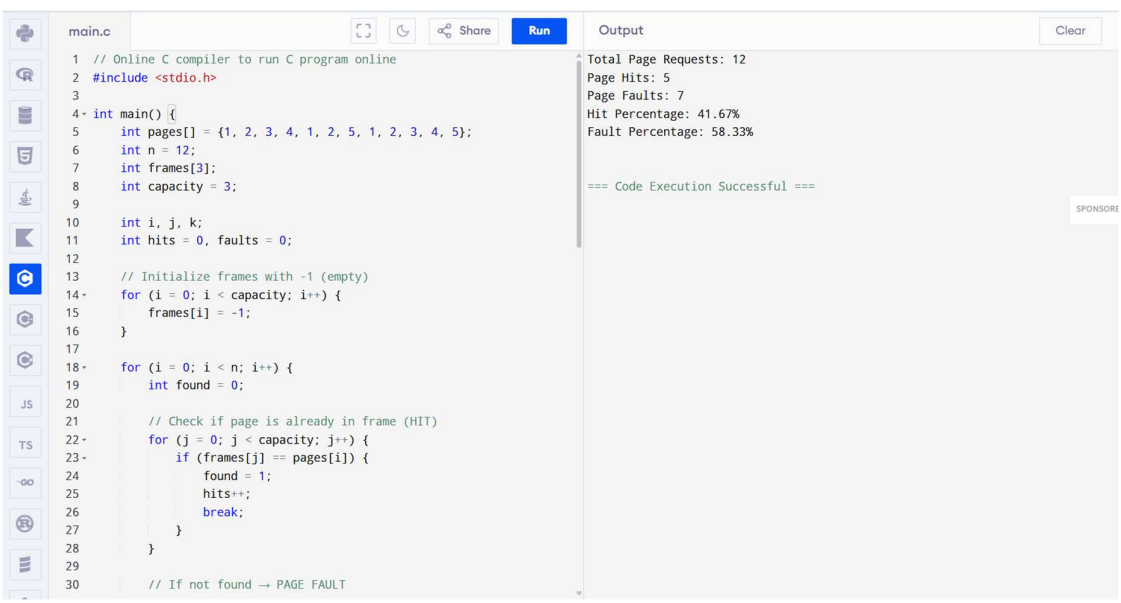
❖ **CODE:**

```
#include <stdio.h>

int main() {
   int pages[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
   int n = 12;
   int frames[3];
   int capacity = 3;

   int i, j, k;
   int hits = 0, faults = 0;

   // Initialize frames with -1 (empty)
   for (i = 0; i < capacity; i++) {
      frames[i] = -1;
   }

   for (i = 0; i < n; i++) {
      int found = 0;

      // Check if page is already in frame (HIT)
      for (j = 0; j < capacity; j++) {
```

```
        if (frames[j] == pages[i]) {
            found = 1;
            hits++;
            break;
        }
    }

    // If not found → PAGE FAULT
    if (!found) {
        int replace_index = -1;

        // Check for empty frame
        for (j = 0; j < capacity; j++) {
            if (frames[j] == -1) {
                replace_index = j;
                break;
            }
        }

        // If no empty frame → apply Optimal Replacement
        if (replace_index == -1) {
            int farthest = -1;

            for (j = 0; j < capacity; j++) {
                int next_use = -1;

                for (k = i + 1; k < n; k++) {
                    if (frames[j] == pages[k]) {
                        next_use = k;
                        break;
                    }
                }

                // If page is not used again
                if (next_use == -1) {
                    replace_index = j;
                    break;
                }

                // Find farthest used page
                if (next_use > farthest) {
                    farthest = next_use;
                    replace_index = j;
                }
```

```
            }
        }

        frames[replace_index] = pages[i];
        faults++;
    }
}

float hit_percentage = (float)hits / n * 100;
float fault_percentage = (float)faults / n * 100;

printf("Total Page Requests: %d\n", n);
printf("Page Hits: %d\n", hits);
printf("Page Faults: %d\n", faults);
printf("Hit Percentage: %.2f%%\n", hit_percentage);
printf("Fault Percentage: %.2f%%\n", fault_percentage);

return 0;
 }
```

❖ **Output:**



❖ **Conclusion:** In this practical, we conclude that the **LRU page replacement algorithm** helps manage memory efficiently by minimizing page faults and improving performance.

❖ **Discussion Questions:**

1. What is the Optimal page replacement algorithm?

2. How do you determine a page hit and a page fault?

3. What is the formula to calculate page hit and page fault percentage?

4. Why is Optimal preferred over FIFO for page replacement?

5. What is the worst-case scenario for Optimal page replacement?

❖ **References:**

https://www.scaler.com/topics/operating-system/page-replacement-algorithm/
https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26